

Desenvolvimento de um Simulador de Auto Battler Integrado ao Treinamento de Inteligência Artificial

Herich Gabriel de Campos¹, Josiel Neumann Kuk¹

¹Departamento de Ciência da Computação
Universidade Estadual do Centro-Oeste (UNICENTRO)
Guarapuava – PR – Brazil

57023740007@unicentro.edu.br, josiel@unicentro.br

Abstract. *This work presents the development of a custom, highly optimized Auto Battler simulator written in C++, coupled with a Python interface via pybind11, specifically designed for training Artificial Intelligence agents. Commercial Auto Battlers feature complex mechanics such as shared unit pools, grid-based pathfinding, and dynamic economy systems, making them challenging environments for machine learning. By creating a lightweight, headless simulator that provides dense rewards and detailed combat statistics, this project enables efficient agent training and evaluation. The methodology covers the architectural design of the simulation engine, including the combat loop, pathfinding algorithms, and the integration pipeline for reinforcement learning algorithms.*

Resumo. *Este trabalho apresenta o desenvolvimento de um simulador otimizado de Auto Battler escrito em C++, integrado a uma interface Python via pybind11, projetado especificamente para o treinamento de agentes de Inteligência Artificial. Auto Battlers comerciais possuem mecânicas complexas, como pools compartilhadas de unidades, busca de rotas em grade e sistemas dinâmicos de economia, tornando-os ambientes desafiadores para aprendizado de máquina. Através da criação de um simulador leve, sem interface gráfica (headless) e capaz de fornecer recompensas densas (dense rewards) e estatísticas detalhadas de combate, este projeto viabiliza o treinamento e avaliação eficientes de agentes. A metodologia abrange o projeto arquitetural do motor de simulação, incluindo o laço de combate, algoritmos de busca de rotas e a esteira de integração para algoritmos de aprendizado por reforço.*

1. Introdução

O gênero de jogos *Auto Battler*, popularizado por títulos como *Teamfight Tactics* — que ultrapassou 33 milhões de jogadores mensais [Henkel 2019] — e *Dota Underlords*, baseia-se em escolhas estratégicas de economia, posicionamento em grade e sinergia de unidades automáticas. Devido ao seu vasto espaço de estados e tomada de decisões com informações imperfeitas, este gênero apresenta um ambiente rico e altamente complexo para a pesquisa em Inteligência Artificial (IA).

Aplicar técnicas de aprendizado de máquina diretamente em clientes comerciais de jogos é muitas vezes inviável devido ao alto custo computacional de renderização e

à ausência de APIs que exponham o estado interno do jogo de forma acelerada. Faz-se necessária, portanto, a construção de motores de simulação *headless* que permitam a execução de milhares de partidas por segundo, à semelhança de ambientes como o OpenAI Gym [Brockman et al. 2016].

O objetivo geral deste trabalho é desenvolver um motor de simulação de Auto Battler de alto desempenho em C++ e utilizá-lo como ambiente para o treinamento de um agente de Inteligência Artificial através de *bindings* em Python. Os objetivos específicos incluem: a implementação de um sistema de combate com busca de rotas em tempo real, mitigação de danos e controle de habilidades; e a estruturação de um sistema de recompensas densas (*Dense Rewards*) para acelerar a convergência do aprendizado do agente.

O restante deste documento está organizado da seguinte forma: a Seção 3 apresenta o referencial teórico; a Seção 4 discute os trabalhos relacionados; a Seção 5 detalha a arquitetura do simulador; a Seção 6 descreve os experimentos realizados; e a Seção 7 apresenta as conclusões e trabalhos futuros.

2. Arquitetura e Desenvolvimento do Simulador

2.1. Arquitetura Híbrida C++ e Python

O simulador foi projetado utilizando uma arquitetura híbrida para aliar máxima performance de processamento à flexibilidade de integração. Toda a lógica computacional intensiva de jogo, incluindo cálculos de dano, mitigação por armadura e resistência mágica, busca de rotas espaciais e instanciamento de habilidades, foi implementada nativamente em C++. Esta abordagem garante eficiência no uso de memória e velocidade de execução, requisitos fundamentais para o treinamento de agentes de Aprendizado por Reforço, cujo sucesso depende da simulação de milhões de cenários em curto intervalo de tempo.

A camada superior de controle foi desenvolvida em Python. Através da biblioteca `pybind11`, o código em C++ foi compilado em um módulo compartilhável, permitindo que scripts em Python inicializem o tabuleiro, aloquem as unidades e gerenciem o fluxo da simulação. Esta separação de responsabilidades cria uma interface padronizada que elimina gargalos de comunicação e facilita a integração futura com bibliotecas de Inteligência Artificial.

2.2. Ingestão Dinâmica de Dados

Para garantir a flexibilidade na configuração dos experimentos, o motor não possui status de entidades ou regras de balanceamento codificadas rigidamente no código. Utilizou-se a biblioteca `nlohmann/json` para a leitura de estruturas externas no formato JSON. Modificações referentes a custos, atributos base, adição ou remoção de classes e itens são realizadas diretamente nestes arquivos, refletindo instantaneamente no ambiente simulado sem a necessidade de recompilação do binário C++. A implementação deste analisador léxico assegura o isolamento de memória, prevenindo comportamentos indefinidos e vazamentos de dados na aplicação.

2.3. O Motor de Combate (*Combat Engine*)

O núcleo da simulação é governado por um laço temporal iterativo baseado em *ticks*. O ambiente é atualizado em frações de 0.1 segundos, momento em que o motor deduz os tempos de recarga de ataque e processa a geração passiva de mana das unidades.

A movimentação e o cálculo de distâncias ocorrem em uma matriz estruturada em uma grade de 7x8. Quando uma entidade não possui um alvo válido em seu alcance de ataque, o sistema emprega o algoritmo de Busca em Largura (BFS) para calcular a rota mais eficiente, orientando o deslocamento pelas células vazias. A cada instância de ataque ou invocação de magia, o motor reavalia o estado do tabuleiro para determinar a unidade adversária viva mais próxima, lidando de forma consistente com mortes em tempo real e readequação de alvos. Adicionalmente, foi implementada uma mecânica de limite de tempo: ao atingir 30 segundos de combate, um gatilho de morte súbita é acionado para acelerar a resolução de impasses.

2.4. Sistema de Dano, Habilidades e Sinergias

A mecânica de combate distingue entre dano bruto, dano real e dano verdadeiro. Todo dano processado pelo motor passa por funções de mitigação que aplicam fórmulas de redução baseadas nas resistências do alvo, com exceção do dano verdadeiro, que ignora as mitigações defensivas. O simulador suporta mecânicas complexas de feitiços, incluindo curas, escudos escaláveis com base nos pontos de vida (HP) máximos, ataques em área de efeito (AoE), dano em cone e atordoamentos temporários.

Para viabilizar a formulação de recompensas no modelo de Inteligência Artificial, o motor implementa rastreamento granular de desempenho. Todo o dano provocado por um campeão é acumulado em variáveis internas da entidade, sendo exportado ao término do combate para integrar a Função de Recompensa (*Reward Function*) da rede neural.

A ativação de sinergias é controlada por estruturas de dados do tipo `std::set`, que validam as instâncias no tabuleiro para garantir que características exclusivas sejam contadas adequadamente. Estas sinergias operam em dois momentos distintos: no pré-combate, aplicando modificadores imediatos na inicialização da rodada (como escudos brutos ou amplificadores de dano), e durante o combate, inserindo eventos específicos diretamente no laço de execução temporal, como atordoamentos globais agendados, conjuração dupla de feitiços ou execução imediata de unidades adversárias.

2.5. Estruturas de Gerenciamento do Metajogo

No nível macro, o ambiente modela não apenas o combate, mas todo o ciclo de vida e progressão de uma partida do gênero. O meta-modelo implementa uma reserva global (*global pool*) de peças disponíveis, limitando a oferta e forçando a concorrência estratégica pelo mesmo conjunto de recursos. O sistema econômico computa o acúmulo de ouro considerando a renda base, rendimentos de juros acumulados e bônus dinâmicos por sequências de vitórias ou derrotas (*streaks*). O gerenciamento de partida abrange ainda a alocação de pontos de experiência, evolução de níveis e o algoritmo de apresentação aleatória e probabilística de unidades na loja a cada ciclo.

3. Referencial Teórico

Para modelar com precisão o ambiente do jogo, é necessário compreender as mecânicas fundamentais dos Auto Battlers. O sistema de combate baseia-se em atributos matemáticos como mitigação de dano por armadura, taxa de ataque baseada em *ticks* de tempo e alcance das unidades. A movimentação no tabuleiro exige algoritmos eficientes de busca, como a Busca em Largura (BFS), para desviar de obstáculos dinâmicos em uma métrica de Distância de Manhattan.

No contexto da Inteligência Artificial, o treinamento de agentes em ambientes complexos é fundamentado pela teoria do Aprendizado por Reforço [Sutton and Barto 2018], na qual um agente aprende uma política de ações ao maximizar uma função de recompensa acumulada ao longo do tempo. A utilização de recompensas densas — onde o agente é bonificado por métricas intermediárias como dano causado ou duração de sobrevivência — em contraposição a recompensas esparsas (apenas vitória ou derrota), é um fator crucial para a viabilidade do treinamento em tempo hábil [Mnih et al. 2015]. Como alternativa ou complemento, Algoritmos Genéticos [Goldberg 1989, Holland 1975] também podem ser empregados na otimização de estratégias de posicionamento.

4. Trabalhos Relacionados

Trabalhos recentes demonstram o crescente interesse em aplicar IA especificamente ao domínio dos Auto Battlers. Liu [Liu 2022] propôs uma abordagem computacional para otimização de posicionamento no *Teamfight Tactics*, utilizando técnicas de busca e avaliação de estados. De forma similar, Zhang [Zhang 2024] desenvolveu um agente inteligente para Auto Battlers com base em aprendizado por reforço, evidenciando a viabilidade de algoritmos como PPO [Schulman et al. 2017] nesse contexto.

O presente trabalho se diferencia ao construir um motor de simulação próprio em C++ de alta performance, em vez de interagir com o cliente do jogo, e ao fornecer estatísticas granulares de combate (dano causado, mitigado e cura) para compor funções de recompensa densas, viabilizando um ciclo de treinamento mais eficiente.

5. Arquitetura e Desenvolvimento do Simulador

5.1. Modelagem de Dados e *Global Pool*

As entidades do jogo, como campeões e seus atributos, foram externalizadas em estruturas de dados padronizadas no formato JSON, facilitando a extensão da base de dados sem recompilação do motor. O gerenciamento de peças disponíveis é feito por meio de uma *Global Pool*, garantindo que a probabilidade de aparição de uma unidade na loja diminua conforme cópias são retiradas do estoque, espelhando a dinâmica de escassez competitiva dos jogos comerciais.

5.2. O Motor de Combate (*Combat Engine*)

O fluxo de combate foi projetado para operar em frações de tempo chamadas *ticks*. A cada pulso, a máquina de estados verifica temporizadores de ataque (*cooldowns*), acúmulo de mana e invocações de habilidades especiais. A movimentação é resolvida dinamicamente por uma rotina BFS otimizada para matrizes bidimensionais, na qual cada unidade busca o hexágono válido mais próximo de seu alvo.

5.3. A Ponte de Integração C++ / Python

Para viabilizar o uso do simulador por bibliotecas modernas de IA (como TensorFlow ou PyTorch), a lógica central em C++ foi exposta para Python através da biblioteca *pybind11*. Esta abordagem permite que a rede neural decida as ações de alto nível no ambiente Python, enquanto toda a carga computacional da simulação de combates ocorre nativamente em C++, eliminando gargalos de comunicação entre as camadas.

6. Experimentos e Resultados

6.1. Avaliação de Performance

Serão realizados testes de desempenho medindo a taxa de partidas completas simuladas por segundo em diferentes configurações de número de unidades, a fim de quantificar o ganho obtido pela implementação nativa em C++ em comparação a abordagens baseadas exclusivamente em Python.

6.2. Treinamento do Agente de IA

Serão conduzidos experimentos de treinamento utilizando o algoritmo PPO [Schulman et al. 2017], com e sem o uso de recompensas densas derivadas das estatísticas de combate do motor. A comparação entre as curvas de aprendizado permitirá avaliar o impacto das recompensas intermediárias na velocidade de convergência do agente.

7. Conclusão e Trabalhos Futuros

Este trabalho propõe o desenvolvimento de um simulador escalável e veloz para o treinamento de agentes de IA no domínio de Auto Battlers. A arquitetura baseada em um motor C++ com interface Python via `pybind11` visa eliminar os principais gargalos computacionais presentes em abordagens que utilizam clientes comerciais, viabilizando ciclos de treinamento mais eficientes.

Como trabalhos futuros, planeja-se a expansão da base de dados de unidades e a inclusão de mecânicas de itens combináveis, o que aumentará exponencialmente o espaço de estados e demandará abordagens mais sofisticadas no modelo da rede neural, como arquiteturas baseadas em atenção (*transformers*).

Referências

- Brockman, G. et al. (2016). OpenAI Gym.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, Reading.
- Henkel, R. (2019). Teamfight tactics has more than 33 million players per month. ONE Esports.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press, Ann Arbor.
- Liu, D. (2022). Computer knows best: A computational approach to positioning in teamfight tactics.
- Mnih, V. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533.
- Schulman, J. et al. (2017). Proximal policy optimization algorithms.
- Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, 2 edition.
- Zhang, W. (2024). Design and implementation of an intelligent agent for auto battler games using reinforcement learning.